

SLOVENSKÁ TECHNICKÁ UNIVERZITA V BRATISLAVE

Fakulta elektrotechniky a informatiky

Vnorený riadiaci systém pre model robotického manipulátora

Dokumentácia

Študijný program: Robotika a kybernetika
Predmet: I-TP-RK – Tímový projekt
Školiteľ: Ing. Martin Dodek, PhD.

**Bc. Simona Walterová, Bc. Samuel Vavruš,
Bc. Samuel Valko, Bc. Samuel Kopp,
Bc. Márk Pšenák, Bc. Filip Dubovský,
Bc. Jakub Janega**

TÍMOVÝ PROJEKT

ZADANIE

Študijný program:

Robotika a kybernetika

Študijný odbor:

kybernetika

Vedúci projektu:

Ing. Martin Dodek, PhD.

Miesto vypracovania projektu:

Ústav robotiky a kybernetiky

Riešitelia:

Bc. Simona Walterová

Bc. Jakub Janega

Bc. Samuel Kopp

Bc. Samuel Vavruš

Bc. Samuel Valko

Bc. Filip Dubovský

Bc. Márk Pšenák

Názov projektu:

Vnorený riadiaci systém pre model robotického manipulátora

Špecifikácia zadania:

Tímový projekt sa zameriava na implementáciu vnoreného riadiaceho systému pre model robotického manipulátora so šiestimi stupňami voľnosti. Projekt využíva mikrokontrolér STM32 a jazyk C++. Úlohou je určiť kinematickú štruktúru manipulátora na základe jeho konštrukcie (geometrie), navrhnuť a implementovať algoritmus numerického riešenia inverznej kinematickej úlohy a zabezpečiť, aby robot dokázal dosiahnuť požadovanú polohu a orientáciu efektora. Celý riadiaci algoritmus bude implementovaný ako plne vnorené riešenie na mikrokontroléri STM32, s využitím knižnice Eigen, prostriedkov operačného systému reálneho času a medziprocesovej komunikácie. Projekt zahŕňa aj návrh a implementáciu používateľského rozhrania (displej) a integráciu ovládacích prvkov (joystick a tlačidlá).

Úlohy:

1. Na základe konštrukcie modelu (geometrie) určiť kinematickú štruktúru robotického manipulátora. Formulovať doprednú a inverznú kinematickú úlohu.
2. Navrhnuť numerické riešenie inverznej kinematiky Newtonovou metódou. Implementovať algoritmus numerického riešenia inverznej kinematiky v jazyku C++ s využitím knižnice Eigen.
3. Implementovať riadiaci softvér na mikrokontroléri STM32 s využitím RTOS. Rozdeliť úlohy na procesy (výpočty, komunikácia, rozhranie) a implementovať prvky medziprocesovej komunikácie.
4. Navrhnuť užívateľské rozhranie pre ovládanie robota (joystick) a zadávanie požadovanej pozície a orientácie efektora.
5. Overiť funkčnosť a vyhodnotiť presnosť dosiahnutia polohy a orientácie.

6. Vypracovať technickú dokumentáciu k implementácii. Zdokumentovať architektúru softvéru, použité algoritmy a implementované triedy.

***Termín odovzdania projektu:* 29.5.2026**

V Bratislave dňa 1.2.2025

**prof. Ing. Jarmila Pavlovičová PhD.
garantka študijného programu**

Obsah

Úvod	5
1 Geometria modelu ramena	6
2 Numerické riešenie	7
2.1 Priama kinematika	7
2.1.1 Pseudokód pre priamu kinematiku	8
2.2 Inverzná kinematika	8
2.2.1 Jakobián polohy	8
2.2.2 Pseudokód pre Jakobián	9
2.2.3 Jakobián orientácie	10
2.2.4 Pseudokód pre výpočet Jakobiánu orientácie	11
2.2.5 Pseudokód pre inverznú kinematiku	12
3 Implementácia softvéru na mikrokontroléri	14
4 Návrh používateľského rozhrania	15
5 Overenie funkčnosti a presnosti polohovania	16
Záver	17

Úvod

Úvod (ak je potrebný)

1 Geometria modelu ramena

2 Numerické riešenie

Keďže náš manipulátor sa skladá s 6 článkov, tak v nasledovných rovniciach budeme uvažovať výpočet pre týchto 6 článkov. Všeobecne však môžeme uvažovať ľubovoľný počet článkov, a teda aj ľubovoľný počet premenných. Pseudo programy na výpočet týchto rovníc budú preto všeobecne vyjadrené.

Pre výpočet budeme potrebovať základné vektory a matice. Budeme uvažovať polohový vektor:

$$\mathbf{p} = \begin{pmatrix} p^x \\ p^y \\ p^z \end{pmatrix} \quad (1)$$

a vektor kĺbových premenných:

$$\boldsymbol{\theta} = \begin{pmatrix} \theta_1 \\ \theta_2 \\ \theta_3 \\ \theta_4 \\ \theta_5 \\ \theta_6 \end{pmatrix} \quad (2)$$

rotačné matice a translačný vektor na výpočet polohy koncového efektora:

$$R_x(\theta) = \begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos(\theta) & -\sin(\theta) \\ 0 & \sin(\theta) & \cos(\theta) \end{pmatrix} \quad (3)$$

$$R_y(\theta) = \begin{pmatrix} \cos(\theta) & 0 & \sin(\theta) \\ 0 & 1 & 0 \\ -\sin(\theta) & 0 & \cos(\theta) \end{pmatrix} \quad (4)$$

$$R_z(\theta) = \begin{pmatrix} \cos(\theta) & -\sin(\theta) & 0 \\ \sin(\theta) & \cos(\theta) & 0 \\ 0 & 0 & 1 \end{pmatrix} \quad (5)$$

$$T = \begin{pmatrix} 0 \\ 0 \\ L \end{pmatrix} \quad (6)$$

2.1 Priama kinematika

Priama kinematika nám umožňuje vypočítať polohu koncového efektora na základe zadaných kĺbových premenných. Medzi jednotlivými súradnicovými systémami platia

transformácie:

$$p_5 = R_6(\theta_6)(T_6 + p_6) \quad (7)$$

$$p_4 = R_5(\theta_5)(T_5 + p_5) \quad (8)$$

$$p_3 = R_4(\theta_4)(T_4 + p_4) \quad (9)$$

$$p_2 = R_3(\theta_3)(T_3 + p_3) \quad (10)$$

$$p_1 = R_2(\theta_2)(T_2 + p_2) \quad (11)$$

$$p_0 = R_1(\theta_1)(T_1 + p_1) \quad (12)$$

Kinematika koncového efektora manipulátora je potom daná ako (transformácia z p_6 do p_0):

$$p_0(\theta) = R_1(\theta_1) \left(T_1 + R_2(\theta_2) \left(T_2 + R_3(\theta_3) \left(T_3 + R_4(\theta_4) \left(T_4 + R_5(\theta_5) \left(T_5 + R_6(\theta_6)(T_6) \right) \right) \right) \right) \right) \quad (13)$$

2.1.1 Pseudokód pre priamu kinematiku

Nižšie je uvedený pseudo program na výpočet priamej kinematickej úlohy, ktorý zohľadňuje všetky články manipulátora a ich vzájomné transformácie. Čiže pseudo program vracia polohový vektor a rotačnú maticu koncového efektora vzhľadom na základňu manipulátora.

```
function forwardKinematics(theta[], R[])
    vekt = 0;
    matica = I_3x3;
    n = size(theta[]);

    for (i = n; i > 0; i--) begin
        vekt = R[i](theta[i]) * (T[i] + vekt);
        matica = R[i](theta[i]) * matica;
    end

    return vekt, matica;
end function
```

Výpis kódu 1: Výpočet priamej kinematickej úlohy

2.2 Inverzná kinematika

2.2.1 Jakobián polohy

Na výpočet inverznej kinematickej úlohy je potrebné poznať Jakobián polohy, ktorý nám umožní vypočítať zmenu polohy koncového efektora vzhľadom na zmenu

kĺbových premenných. Jakobiánu polohového vektora $p_0(\theta)$ možno definovať ako maticu parciálnych derivácií $\nabla p_0(\theta) \in \mathbb{R}^{3 \times 6}$:

$$\nabla p_0(\theta) = \begin{pmatrix} \frac{\partial p_0(\theta)}{\partial \theta_1} & \frac{\partial p_0(\theta)}{\partial \theta_2} & \frac{\partial p_0(\theta)}{\partial \theta_3} & \frac{\partial p_0(\theta)}{\partial \theta_4} & \frac{\partial p_0(\theta)}{\partial \theta_5} & \frac{\partial p_0(\theta)}{\partial \theta_6} \end{pmatrix} \quad (14)$$

Na výpočet matice budeme potrebovať vypočítať parciálne derivácie polohového vektora $p_0(\theta)$ vzhľadom na jednotlivé kĺbové premenné θ_i .

$$\frac{\partial p_0(\theta)}{\partial \theta_1} = \frac{\partial(R_1(\theta_1))}{\partial \theta_1} \left(T_1 + R_2(\theta_2) \left(T_2 + R_3(\theta_3) \left(T_3 + R_4(\theta_4) (T_4 + R_5(\theta_5) (T_5 + R_6(\theta_6)(T_6))) \right) \right) \right) \quad (15)$$

$$\frac{\partial p_0(\theta)}{\partial \theta_2} = R_1(\theta_1) \left(\frac{\partial(R_2(\theta_2))}{\partial \theta_2} \left(T_2 + R_3(\theta_3) \left(T_3 + R_4(\theta_4) (T_4 + R_5(\theta_5) (T_5 + R_6(\theta_6)(T_6))) \right) \right) \right) \quad (16)$$

$$\frac{\partial p_0(\theta)}{\partial \theta_3} = R_1(\theta_1) R_2(\theta_2) \left(\frac{\partial(R_3(\theta_3))}{\partial \theta_3} \left(T_3 + R_4(\theta_4) (T_4 + R_5(\theta_5) (T_5 + R_6(\theta_6)(T_6))) \right) \right) \quad (17)$$

$$\frac{\partial p_0(\theta)}{\partial \theta_4} = R_1(\theta_1) R_2(\theta_2) R_3(\theta_3) \left(\frac{\partial(R_4(\theta_4))}{\partial \theta_4} \left(T_4 + R_5(\theta_5) (T_5 + R_6(\theta_6)(T_6)) \right) \right) \quad (18)$$

$$\frac{\partial p_0(\theta)}{\partial \theta_5} = R_1(\theta_1) R_2(\theta_2) R_3(\theta_3) R_4(\theta_4) \left(\frac{\partial(R_5(\theta_5))}{\partial \theta_5} \left(T_5 + R_6(\theta_6)(T_6) \right) \right) \quad (19)$$

$$\frac{\partial p_0(\theta)}{\partial \theta_6} = R_1(\theta_1) R_2(\theta_2) R_3(\theta_3) R_4(\theta_4) R_5(\theta_5) \frac{\partial(R_6(\theta_6))}{\partial \theta_6} (T_6) \quad (20)$$

2.2.2 Pseudokód pre Jakobián

Nižšie je uvedený pseudo program na výpočet Jakobiho matice, ktorý zohľadňuje všetky články manipulátora a ich vzájomné transformácie. Tento pseudo program nám umožní vypočítať Jakobián polohy efektora vzhľadom na kĺbové premenné.

```
function jacobian(theta[], R[], dR[])
    n = size(theta[]);
    jacob_mat = 3 x n;
    matica = I_3x3;
    pom_vekt = T[n];

    for(i = n-1; i > 0; i--)
        begin
```

```

    matica = R[i](theta[i]) * matica;
end

for(j = n; j > 0; j--)
begin
    vekt = T[n];
    if(n > j)
    begin
        matica = matica * R[j](theta[j]) * transpose();
        pom_vekt = T[j] + R[j+1](theta[j+1]) * pom_vekt;
        vekt = pom_vekt;
    end

    vekt = matica + dR[j](theta[j]) * vekt;
    jacob_mat.col(j) = vekt;
end

return jacob_mat;
end function

```

Výpis kódu 2: Výpočet Jakobiho matice

2.2.3 Jakobián orientácie

Rotačné matice jednotlivých článkov robota tvoria celkovú rotačnú maticu pre súradnicový systém efektora, ktorú získame z funkcie priamej kinematiky. Táto matica nám umožní vypočítať orientáciu efektora vzhľadom na základňu manipulátora. Celková rotačná matica $R_T(\theta)$ je daná súčinom rotačných matíc jednotlivých článkov:

$$R_T(\theta) = R_1(\theta_1)R_2(\theta_2)R_3(\theta_3)R_4(\theta_4)R_5(\theta_5)R_6(\theta_6) \quad (21)$$

Následne vypočítame derivácie $R_T(\theta)$ pre získanie rotačného Jakobiánu, ktorý nám umožní vypočítať zmenu orientácie efektora vzhľadom na zmenu kľbových premenných:

$$\frac{\partial R_T(\theta)}{\partial \theta_1} = \frac{\partial R_1(\theta_1)}{\partial \theta_1} R_2(\theta_2) R_3(\theta_3) R_4(\theta_4) R_5(\theta_5) R_6(\theta_6) \quad (22)$$

$$\frac{\partial R_T(\theta)}{\partial \theta_2} = R_1(\theta_1) \frac{\partial R_2(\theta_2)}{\partial \theta_2} R_3(\theta_3) R_4(\theta_4) R_5(\theta_5) R_6(\theta_6) \quad (23)$$

$$\frac{\partial R_T(\theta)}{\partial \theta_3} = R_1(\theta_1) R_2(\theta_2) \frac{\partial R_3(\theta_3)}{\partial \theta_3} R_4(\theta_4) R_5(\theta_5) R_6(\theta_6) \quad (24)$$

$$\frac{\partial R_T(\theta)}{\partial \theta_4} = R_1(\theta_1) R_2(\theta_2) R_3(\theta_3) \frac{\partial R_4(\theta_4)}{\partial \theta_4} R_5(\theta_5) R_6(\theta_6) \quad (25)$$

$$\frac{\partial R_T(\theta)}{\partial \theta_5} = R_1(\theta_1) R_2(\theta_2) R_3(\theta_3) R_4(\theta_4) \frac{\partial R_5(\theta_5)}{\partial \theta_5} R_6(\theta_6) \quad (26)$$

$$\frac{\partial R_T(\theta)}{\partial \theta_6} = R_1(\theta_1)R_2(\theta_2)R_3(\theta_3)R_4(\theta_4)R_5(\theta_5)\frac{\partial R_6(\theta_6)}{\partial \theta_6} \quad (27)$$

Reziduum (chybu) pre polohu efektora vyjadríme ako:

$$r_p(\theta) = p_0(\theta) - p_r \quad (28)$$

Chceme dosiahnuť, aby veľkosť tejto chyby bola minimálna, čo znamená že polohový vektor efektora sa zhoduje s požadovaným polohovým vektorom p_r .

2.2.4 Pseudokód pre výpočet Jakobiánu orientácie

Nižšie je uvedený pseudo program na výpočet derivácií rotačných matíc, ktorý zohľadňuje všetky články manipulátora a ich vzájomné transformácie.

```
function derRotMat(theta[], dR[], R[])
    n = size(theta[]);
    mat_all(n);
    f_mat = I_3x3;
    b_mat = f_mat;

    for(i = n-1; i > 0; i--)
        begin
            f_mat = R[i](theta[i]) * matica;
        end

    for(j = n; j > 0; j--)
        begin
            if(n > j)
                begin
                    b_mat = R[j+1](theta[j+1]) * b_mat;
                    f_mat = f_mat * R[j](theta[j]).transpose();
                end
            mat_all(j) = f_mat * dR[j](theta[j]) * b_mat;
        end

    return mat_all;
end function
```

Výpis kódu 3: Výpočet derivácií rotačných matíc

Reziduum (chybu) pre orientáciu efektora vyjadríme ako:

$$r_o(\theta) = \begin{pmatrix} (0 \ 1 \ 0)(R_T(\theta)(R_{RPY}(\theta_R, \theta_P, \theta_Y))^T) \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix} \\ (0 \ 0 \ 1)(R_T(\theta)(R_{RPY}(\theta_R, \theta_P, \theta_Y))^T) \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix} \\ (0 \ 0 \ 1)(R_T(\theta)(R_{RPY}(\theta_R, \theta_P, \theta_Y))^T) \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix} \end{pmatrix} \quad (29)$$

Náš cieľ je dosiahnuť, aby obidve rezidua boli nulové (resp. minimálne), čo znamená že polohový vektor a orientácia efektora sa zhodujú s požadovanými hodnotami p_r a $R_{RPY}(\theta_R, \theta_P, \theta_Y)$.

$$r_p(\theta) = 0 \quad (30)$$

$$r_o(\theta) = 0 \quad (31)$$

Newtonova metóda bude iteratívne aktualizovať vektor kľbových premenných θ na základe rezidiu a Jakobiánu, aby sa minimalizovali chyby polohy a orientácie efektora.

$$\left(\begin{pmatrix} \frac{\partial p_o(\theta)}{\partial \theta_1} & \frac{\partial p_o(\theta)}{\partial \theta_2} & \frac{\partial p_o(\theta)}{\partial \theta_3} & \frac{\partial p_o(\theta)}{\partial \theta_4} & \frac{\partial p_o(\theta)}{\partial \theta_5} & \frac{\partial p_o(\theta)}{\partial \theta_6} \end{pmatrix} \begin{pmatrix} 1 \\ 0 \\ 0 \\ 1 \\ 0 \\ 0 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \\ 0 \\ 1 \\ 0 \\ 0 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \\ 0 \\ 1 \\ 0 \\ 0 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \\ 0 \\ 1 \\ 0 \\ 0 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \\ 0 \\ 1 \\ 0 \\ 0 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \\ 0 \\ 1 \\ 0 \\ 0 \end{pmatrix} \right)_{\theta=\theta(k)} + \Delta \theta(k) = \begin{pmatrix} -r_p(\theta(k)) \\ -r_o(\theta(k)) \end{pmatrix} \quad (32)$$

$$\theta(k+1) = \theta(k) + \Delta \theta(k) \quad (33)$$

Algoritmus sa ukončí ak je splnená podmienka (veľkosť rezidua):

$$(r_p(\theta))^T(r_p(\theta)) + (r_o(\theta))^T(r_o(\theta)) < \epsilon \quad (34)$$

2.2.5 Pseudokód pre inverznú kinematiku

Nížšie je uvedený pseudo program na iteratívny výpočet inverznej kinematiky pomocou Newtonovej metódy, ktorý zohľadňuje všetky články manipulátora a ich vzájomné transformácie.

```
function solveIK(pr, Rr, theta, tol_pos, tol_ori, max_iter)
    for(i=0; i < max_iter; i++)
        begin
```

```

[p0, R0] = forwardKinematics(theta, R);
(3,3) rp = p0 - Pr;
(3,3) E = R0 + Rr.transpose();
ro(0,0) = E(1,0);           // (010) * E(100) ~T
ro(1,0) = E(2,0);           // (001) * E(100) ~T
ro(2,0) = E(2,1);           // (001) * E(010) ~T

if(rp.norm < tol_pos && ro.norm < tol_ori)
begin
    return 1;                // úspešne max_iterov
end

J_p = jacobian(theta, R, dR);
der_R = derRotMat(theta, dR, R);
J.block<3,6>(0,0) = J_p.block<3,6>(0,0);

for(j=0; j < DOF; ++j)
begin
    (3,3) dE_j = der_R[j] * Rr.transpose();

    J(3,j) = dE_j(1,0);
    J(4,j) = dE_j(2,0);
    J(5,j) = dE_j(2,1);
end

for(k=0; k < DOF; ++k)
begin
    J(k,k) += lambda;
end

minus_r.block<3,1>(0,0) = -rp;
minus_r.block<3,1>(3,0) = -ro;
delta_theta = vyries_sustavu_6x6(J, minus_r);
theta = theta + delta_theta;
end
return false;
end function

```

Výpis kódu 4: Iteratívny výpočet inverznej kinematiky

3 Implementácia softvéru na mikrokontroléri

4 Návrh používateľského rozhrania

5 Overenie funkčnosti a presnosti polohovania

Záver

Záver (ak je potrebný)